

# O Codificador Limpo:

Um Código de Conduta para Programadores Profissionais



Capítulo 7: Teste de Aceitação

**Este material foi desenvolvido pelo grupo  
PET/ADS do IFSP São Carlos**

# O “Tradutor” de código

Além de codificar, o programador tem outra tarefa crucial: **compreender** e **ser compreendido** por stakeholders e clientes.

Grande parte dos erros são causados por **má comunicação** : programadores que não entendem o negócio e *stakeholders* que não entendem o código.

# Princípio da Incerteza

O pessoal de negócios e os desenvolvedores tentam definir detalhes exatos antes mesmo do projeto começar, o que é impossível e gera desperdício.

Ver o projeto em funcionamento é essencial para que o cliente entenda o que realmente precisa.

Definições ambíguas e argumentos não resolvidos geram retrabalho: o programador profissional deve remover qualquer ambiguidade antes de implementar um projeto.

# O que é “acabar”?

Uma fonte notável de erros de comunicação é o termo “acabado”. Ele pode ter diversos significados, como:

- Projeto finalizado e pronto para entrega;
- Projeto pronto para revisão;
- Projeto feito, mas não testado;

Programadores profissionais têm um único significado para a palavra: código **escrito, testado, revisado e aprovado** pelos *stakeholders*.

# Automação de testes

Testes de aceitação sempre devem ser automatizados: o custo de testes automáticos são minúsculos em comparação aos testes manuais.

Criar testes de aceitação não é “trabalho extra”, já que esses testes servem como especificações para o sistema.

# Testes: quem escreve?

Idealmente, testes de aceitação deveriam ser escritos por todos os envolvidos no projeto, especialmente os *stakeholders*.

Infelizmente, *stakeholders* raramente se envolvem com essa parte do processo de desenvolvimento, deixando a criação de testes para analistas e desenvolvedores.

Enquanto as pessoas de negócios escrevem o “caminho feliz” dos testes, a equipe de garantia de qualidade foca no “caminho triste”, testando o que pode dar errado.

# Testes: quando escrever?

Segundo o princípio da precisão tardia, esses testes devem ser escritos o mais tarde possível. Isso ajuda a evitar a criação de testes que, futuramente, se tornarão obsoletos.

Ainda assim, testes de aceitação precisam ser escritos antes do início da codificação, com os primeiros testes prontos no primeiro dia de iteração.

É imperativo que todos os testes estejam prontos antes do meio da iteração, assim evitando possíveis erros e retrabalho nas etapas finais do projeto.

# Testes de aceitação vs. unidade

Testes de unidade são escritos por programadores para descrever a estrutura e o comportamento técnico de baixo nível, voltado para os próprios programadores.

Já os testes de aceitação são escritos pelo negócio para o negócio, especificando requisitos sob o ponto de vista do utilizador.

Mesmo que sejam testes, tanto testes de unidade quanto os de aceitação têm como principal funcionalidade servir como um documento formal a respeito do comportamento do sistema.

# Testes de... Interface?

Criar especificações para UIs é um trabalho difícil, dado que design visual é subjetivo e volátil, estando sob constante mudança e experimentação.

O Princípio da Responsabilidade única estabelece que coisas que mudam por motivos diferentes devem ser separadas, assim como agrupar coisas que mudam por um motivo em comum.

Enquanto o layout e o formato da UI podem mudar por razões estéticas, essas mudanças não devem afetar o funcionamento subjacente da interface.

# Estado de emergência

Todos os testes do sistema devem correr várias vezes ao dia, disparados automaticamente a cada “commit” de código. Falhas de teste devem ser consideradas como emergências.

Em caso de falha em um desses testes, toda a equipe deve virar toda sua atenção para fazer com que os testes passem. Um teste falhando deixado de lado pode se tornar um grande problema.

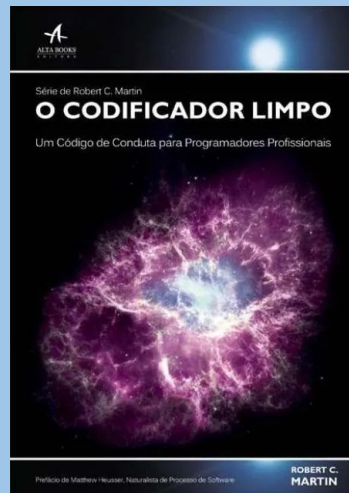
# Sobre o autor

**Steve Krug** conseguiu trabalhar alegremente em completo anonimato como um bastante respeitado consultor de usabilidade até a publicação da primeira edição de Não Me Faça Pensar. Seus livros têm como base sua experiência de duas décadas de trabalho para empresas como Apple, AOL, Bloomberg, Netscape, Lexus, Barnes and Noble e o Fundo Monetário Internacional (FMI).

---

Este conjunto de slides foi elaborado a partir da obra:

MARTIN, Robert C. **O codificador limpo:**  
Um código de conduta para programadores  
profissionais. 1. ed. Rio de Janeiro: Alta  
Books, 2012.



# A equipe



Dylan Petrucelli, Autor

---

Aluno do sexto período, é bolsista do PET/ADS desde fevereiro de 2025.



Gabriel Ribeiro, Revisor

---

Aluno do segundo período, é bolsista do PET/ADS desde novembro de 2025.



Lucas Oliveira, Tutor

---

Professor de Computação, é tutor do PET/ADS desde janeiro de 2023.